BridgeIR: An Agent-Operated, Runtime-Extensible Multi-Backend IR Stack

Hocky Yudhiono
(draft — authors TBD)

October 13, 2026

Abstract

Draft. BridgeIR is an intermediate representation and compiler framework designed for agent-driven compilation. Rather than operating agents on IRs built for human pass authors, we redesign the IR itself around agent affordances: stable node identity that survives transformations and serialization, definitions as runtime data, transactional deltas with provenance, and a canonical, token-lean text format. We ground the design in a survey of production and research IRs across language, tensor, hardware, and database compilers, which shows that expressiveness across IR forms is largely equivalent and that identity, data-defined definitions, and provenance are the unclaimed axes. **TODO:** sharpen thesis, state contributions, results.

1 Introduction

Compiler pipelines are increasingly built and operated by automated agents, yet the frameworks they must extend were designed for humans editing C++ and waiting for rebuilds. Adding a pass, an operation definition, a verifier, or an invariant to an MLIR-based stack today means editing TableGen and C++ sources, rebuilding the tool, and restarting the process. That authoring loop — not the IR data structure — is the bottleneck.

Our thesis is that agent-driven compilation needs an IR designed for agents, not agents bolted onto an IR designed for human pass authors. MLIR and LLVM are excellent at what they target, but they make the opposite choices on the axes that matter here: definitions are build-time code, values are renumbered on every print, and transformations leave no provenance. We therefore design the IR itself around agent affordances — stable identity, data-defined definitions, transactional deltas with provenance — and keep interop with existing ecosystems as optional adapters.

Contributions (draft).

- A tiered extension model for agent-authored compiler extensions — from pipeline manifests and interpreted transformations up to runtime operation definitions and compiled plugins — with precise rebuild boundaries. **TODO**
- A runtime control plane: a manifest registry, plugin loader, and structured (machine-readable) diagnostics. **TODO**
- Invariants as data: per-stage contracts checked at runtime. **TODO:** decide SMT-checked vs. translation-validation depth.
- An evaluation protocol for *agent operability*: task success, time and cost to first successful extension, error recovery, versus a vanilla out-of-tree MLIR workflow. **TODO:** define tasks, baselines, metrics.

Form	Exemplars	Identity	Verification
Linear 3-address	GIMPLE [23]	implicit indices	pass-local checks
CFG + SSA	LLVM [18], MIR [34], SIL [5]	renumbered on print	verifier functions
Sea of nodes	C2, Graal [9, 38]	in-memory pointers	structural invariants
Functional, let-bound	GHC Core/STG [28], Core Erlang [7]	binding names	type systems
Name-keyed graph	ONNX [26], TF GraphDef	**stable names**	schema/shape checks
Loop + schedule	Halide [31], TensorIR [42]	names	shape checks
Structured bytecode	wasm [13], SPIR-V [33]	indices	load-time validation
E-graph	egg [37], TASO [15]	hash-consed classes	rule-based

Table 1: IR forms and their affordances. Expressiveness is largely equivalent across forms; identity stability, definition mechanics, verification, and provenance differ — and those differences are what agents feel.

2 Background: The Design Space of Compiler IRs

Intermediate representations are the interface through which compiler components — frontends, transformations, verifiers, and backends — cooperate. We surveyed the IRs of production and research compilers across language, tensor, hardware, verification, and database domains. Two structural observations motivate this paper.

Chains, not single IRs. Every mature compiler we surveyed is a *chain* of IRs, each tuned to a stage: GCC lowers GENERIC trees to three-address GIMPLE and then to register-transfer lists [23]; GHC goes from typed Core through STG to Cmm [28]; Rust passes through HIR, MIR, and LLVM IR [34]; Swift stages SIL before LLVM [5]; JavaScript engines carry bytecode plus two or more optimizing graph IRs. A framework for agent-driven compilation should therefore treat IR *forms* as schemas and support chains natively, rather than betting on one representation.

Forms are affordances, not expressiveness. The classical forms are largely inter-translatable: CPS, ANF, and SSA are known to correspond [4, 17], and sea-of-nodes IRs make scheduling implicit where CFG-based IRs make it explicit [9, 38]. Expressiveness therefore does not discriminate among forms. What differs is what each form makes cheap or awkward — identity stability, definition mechanics, verification strength, and provenance (Table 1) — precisely the properties that matter when the "user" of the compiler is an automated agent rather than a C++ pass author.

Identity. Optimizing IRs in every surveyed production compiler renumber their values on output: LLVM and MLIR print dense `%0`, `%1`, ... indices, MIR and SIL numbering shifts across passes, and bytecode indices move with their tables. Only *exchange* formats designed for serialization keep names — TensorFlow's GraphDef and ONNX reference values by persistent string identifiers [26] — and those formats carry no optimization infrastructure. Renumbering is costly for an agent: textual

references do not survive a pass boundary, diffs are noisy, and cached observations about particular nodes are invalidated by unrelated edits.

Definitions. Operator definitions are overwhelmingly build-time artifacts: TableGen and ODS generate C++ classes for LLVM and MLIR, Swift uses `.def` files, and JVM bytecode semantics are fixed by specification [20]. Two systems deviate in opposite directions: ONNX defines operators as versioned data schemas with type constraints [26] but has no transformation engine, while TVM registers operators at compiler runtime [8] inside a schedule-driven compiler. No surveyed system combines data-defined definitions with first-class transformations and verification.

Verification. Strong verification precedents exist — the JVM bytecode verifier [20], WebAssembly's load-time type validation [13], eBPF's static safety verifier [21], SPIR-V validation [33], SIL's per-stage verification [5], and nanopass's automatically generated per-stage verifiers [16] — but all are static: they run at load time or build time, none per transformation, and none are transactional.

Provenance. Compiler IRs largely omit provenance; LLVM's optimization remarks attribute transformations to source lines only weakly. By contrast, database optimizers maintain a memo of equivalent expressions and remember which rewrite rules derived each expression [12]; this is the closest existing analogue of the delta-and-journal model we adopt. Identity, data-defined definitions, and provenance are the axes on which BridgeIR positions itself.

3 Positioning: Where MLIR Ends

We frame the design space as a set of axes for MLIR [19]. For each axis we ask what is available today and where systems that go *beyond* it live. "Bigger" here does not mean a single system that dominates MLIR — no such system exists today — but rather the research or engineering direction that pushes that particular axis past what MLIR offers. Table 2 states the comparison.

Reading the table: the first two rows are MLIR's strongest ground — the baseline for human-authored compiler engineering — and under the own-core decision they become interop concerns rather than foundations. BridgeIR's claims live in the middle rows, where it builds agent-first replacements: stable identity instead of renumbering, runtime data-defined schemas instead of build-time ODS, transactional deltas with provenance instead of opaque passes. The last rows we adopt as discipline (versioned schemas, stable serialization) and per-backend work (schedule and hardware semantics). **TODO:** refresh the per-axis "BridgeIR stance" once the design section stabilizes.

4 Related Work

Compiler IRs. LLVM [18] and GCC's GIMPLE [23] established CFG+SSA as the production mid-end form; Swift's SIL [5] adds ownership and staged canonicalization; Rust's MIR [34] is designed for borrow-checking and interpreted by Miri, demonstrating IR-as-executable. Functional compilers contribute typed cores and the many-small-IRs discipline: GHC Core/STG [28], Core Erlang [7], and the nanopass framework [16], which auto-generates a verifier for every tiny stage language. BridgeIR borrows the small-stage philosophy but makes stage definitions and their checks data, and keeps node identity stable across stages.

Axis	MLIR today	Where "bigger" lives
Multi-level structure	Top form: a metalanguage; multi-level IRs are cheap	— (no rival supersedes it)
Declarative definitions (ops/types/attrs)	Top form: ODS/TableGen generates builders, accessors, verifiers	—
Interpreted transformations	Young: PDL patterns and transform-dialect scripts	E-graph rewriting: egg [37], egglog [40]; tensor superoptimization: TASO [15]
Rewrite search and scheduling	Greedy, order-sensitive; fixpoints via ad-hoc pass orders	Equality saturation; relational e-matching [39]; rule inference [25]
Per-compilation specialization	Static pipelines; passes fixed at build time; schedules left to backends	Autotuned schedules (TVM [8], Triton [36]); peephole synthesis (STOKE [32]); generative compilation (SPIRAL [29]); staged specialization (Futamura [10])
Semantics and guarantees	Local per-op C++ verifiers; no formal semantics	Translation validation (Alive2) [22]; mechanized semantics (Vellvm) [41]; SMT-backed contracts
Incrementality	Whole-pass recomputation	Demand-driven compilation: Adapton [14], Salsa [35]
Runtime extensibility	Second-class: `ExtensibleDialect` with C++ verify callbacks; `.so` plugins with exact-ABI requirements	Data-first extension registries; sandboxed plugin hosts (WASM)
Agent operability	Weak: extension means editing `.td`/C++ and rebuilding	Plan → build → load loops; machine-readable pass/dialect descriptions; structured diagnostics
Stability and interchange	No core stability; dialect-level fixes (StableHLO) [27]	Versioned dialects with serialization discipline
Schedule and hardware semantics	Partial: affine, linalg, CIRCT	Schedule-first IRs: Halide [31], TVM TensorIR [42], ISL schedule trees

Table 2: Axis-by-axis comparison. "Bigger" names the direction that pushes an axis past MLIR, not a single competing system.

ML compilers and tensor IRs. XLA's HLO and StableHLO [27] treat stability as an explicit feature; TVM [8] and TensorIR [42] make schedules first-class programs; Triton encodes hardware layouts in IR types [36]; Halide separates algorithm from schedule [31]; PyTorch's FX [30] keeps the graph as data structures in the host language. TASO [15], egg [37], egglog [40], Ruler [25], and relational e-matching [39] contribute equivalence-based rewriting and rule inference. BridgeIR adopts schedules-as-data and equality-friendly rewriting, but targets whole-pipeline agent operation rather than single-kernel optimization.

Verification and provenance. Verification-oriented IRs — Boogie and its descendants [6] — place contracts at the center of the representation. Production IR verifiers (JVM [20], wasm [13], eBPF [21], SPIR-V [33]) demonstrate strong static checking precedents, and database optimizers show how to record derivation history [12]. BridgeIR lifts both ideas to every transformation: transactional commits with invariant gates, and deltas with provenance as first-class values.

Agent-driven compilation. Recent work pairs LLM agents with compilers: AlphaEvolve [11] evolves kernels and algorithms with verification loops; KernelBench [3] benchmarks LLM kernel generation; Meta's LLM Compiler [24] fine-tunes models for LLVM IR tasks; ComPilot [1] assists compiler debugging; Compiler-R1 [2] performs reinforcement-learning-driven optimization tuning. All of these operate on IRs designed for human pass authors. To our knowledge, no system redesigns the IR itself around agent affordances.

Positioning. BridgeIR combines four elements we did not find together in any surveyed system: stable node identity that survives passes and serialization; definitions as runtime data; transactional deltas with provenance; and a canonical, token-lean text format. Each element has precedents — exchange-format naming, ONNX schemas, TVM runtime op registration, Cascades-style derivation tracking, WAT/PTX text canons — but their conjunction, targeted at agents, is new.

References

[1] ComPilot: Llm-assisted compiler debugging and test generation, 2024. Verify title, authors, and venue before citing.

[2] Compiler-R1: Toward agentic compiler auto-tuning with reinforcement learning, 2025. Verify title, authors, and venue before citing.

[3] KernelBench: Can LLMs write efficient GPU kernels?, 2025. Verify authors and venue before citing.

[4] Andrew W. Appel. SSA is functional programming. *ACM SIGPLAN Notices*, 33(4):17–20, 1998.

[5] Apple Inc. Swift intermediate language (SIL). `https://github.com/swiftlang/swift/blob/main/docs/SIL.rst`. Accessed 2026; verify.

[6] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. Boogie: A modular reusable verifier for object-oriented programs. In *Formal Methods for Components and Objects (FMCO)*, 2005.

[7] Richard Carlsson, Björn Gustavsson, Erik Johansson, Thomas Lindgren, Sven-Olof Nyström, Mikael Pettersson, and Robert Virding. Core Erlang 1.0.3 language specification. `https://www.it.uu.se/research/group/hipe/cerl/`. Accessed 2026; verify.

[8] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[9] Cliff Click. A simple graph-based intermediate representation. In *Papers from the 1995 ACM SIGPLAN Workshop on Intermediate Representations (IR)*, 1995.

[10] Yoshihiko Futamura. Partial evaluation of computation process — an approach to a compiler-compiler. *Systems, Computers, Controls*, 2(5):45–50, 1971.

[11] Google DeepMind. AlphaEvolve: A coding agent for designing advanced algorithms, 2025. Verify details before citing.

[12] Goetz Graefe. The Cascades framework for query optimization. *IEEE Data Engineering Bulletin*, 18(3):19–29, 1995.

[13] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, JF Bastien, and Jacques Pienaar. Bringing the web up to speed with WebAssembly. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017.

[14] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. Adapton: Composable demand-driven incremental computation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014.

[15] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. TASO: Optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[16] Andrew W. Keep and R. Kent Dybvig. A nanopass framework for commercial compiler development. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2013.

[17] Richard A. Kelsey. A correspondence between continuation passing style and static single assignment form. In *Papers from the 1995 ACM SIGPLAN Workshop on Intermediate Representations (IR)*, 1995.

[18] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2004.

[19] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021.

[20] Tim Lindholm, Frank Yellin, Gilad Bracha, and Alex Buckley. The Java virtual machine specification. `https://docs.oracle.com/javase/specs/`. Accessed 2026.

[21] Linux Kernel Documentation. BPF verifier. `https://docs.kernel.org/bpf/verifier.html`. Accessed 2026.

[22] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2021.

[23] Jason Merrill. Generic and GIMPLE: A new tree representation for entire functions. In *Proceedings of the GCC Developers Summit*, 2003.

[24] Meta AI. Meta large language model compiler: Foundation models of compiler optimization, 2024. Verify details before citing.

[25] Chandrakana Nandi, Max Willsey, Amy Zhu, Yisu Remy Wang, Brett Saiki, Adam Anderson, Adriana Schulz, Dan Grossman, and Zachary Tatlock. Rewrite rule inference using equality saturation. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2021.

[26] ONNX Contributors. ONNX: Open neural network exchange — operator schemas and specification. `https://github.com/onnx/onnx`. Accessed 2026.

[27] OpenXLA Community. StableHLO: A stable HLO dialect. `https://github.com/openxla/stablehlo`. Accessed 2026.

[28] Simon L. Peyton Jones. Implementing lazy functional languages on stock hardware: The spineless tagless G-machine. *Journal of Functional Programming*, 2(2):127–202, 1992.

[29] Markus Püschel, José M. F. Moura, Jeremy R. Johnson, David Padua, Manuela M. Veloso, Bryan Singer, Jianxin Xiong, Franz Franchetti, Aca Gačić, Yevgen Voronenko, Kang Chen, Robert W. Johnson, and Nicholas Rizzolo. SPIRAL: Code generation for DSP transforms. *Proceedings of the IEEE*, 93(2):232–275, 2005.

[30] PyTorch Team. torch.fx: Practical program capture and transformation. `https://pytorch.org/docs/stable/fx.html`. Accessed 2026.

[31] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013.

[32] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013.

[33] The Khronos Group. The SPIR-V specification. `https://registry.khronos.org/SPIR-V/`. Accessed 2026.

[34] The Rust Project. MIR (mid-level IR). `https://rustc-dev-guide.rust-lang.org/mir/index.html`, . Accessed 2026.

[35] The Rust Project. Salsa: Incremental compilation for Rust. `https://github.com/salsa-rs/salsa`, . Accessed 2026.

[36] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.

[37] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. In *Proceedings of the ACM on Programming Languages (POPL)*, volume 5, 2021.

[38] Thomas Würthinger, Christian Wimmer, Christian Humer, Andreas Wöß, Lukas Stadler, Chris Seaton, Gilles Duboscq, Doug Simon, and Matthias Grimmer. Practical partial evaluation for high-performance dynamic language runtimes. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017.

[39] Yihong Zhang, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. Relational e-matching. In *Proceedings of the ACM on Programming Languages (POPL)*, 2022.

[40] Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. Better together: Unifying datalog and equality saturation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2023.

[41] Jianzhou Zhao, Santosh Nagarakatte, Milo M. K. Martin, and Steve Zdancewic. Formalizing the LLVM intermediate representation for verified program transformations. In *Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2012.

[42] Lianmin Zheng, Ruochen Liu, Junru Shao, Tianqi Chen, Joseph E. Gonzalez, Ion Stoica, and Ameer Haj-Ali. TensorIR: An abstraction for automatic tensorized program optimization. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.